ONLINE HOTEL BOOKING SYSTEM

1. **Title Page:**

INSTITUTE OF ENGINEERING AND MANAGEMENT

COURSE NAME: ADVANCED PROGRAMMING (OOP)

PROJECT TITLE: HOTEL BOOKING MANAGEMENT SYSTEM

STUDENT NAME: PRIYAM DEY

ROLL NUMBER: 51

ENROLLMENT NUMBER: 12024052017053

INSTRUCTOR NAME: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

SEMESTER: SPRING 2026

2. Problem Statement:

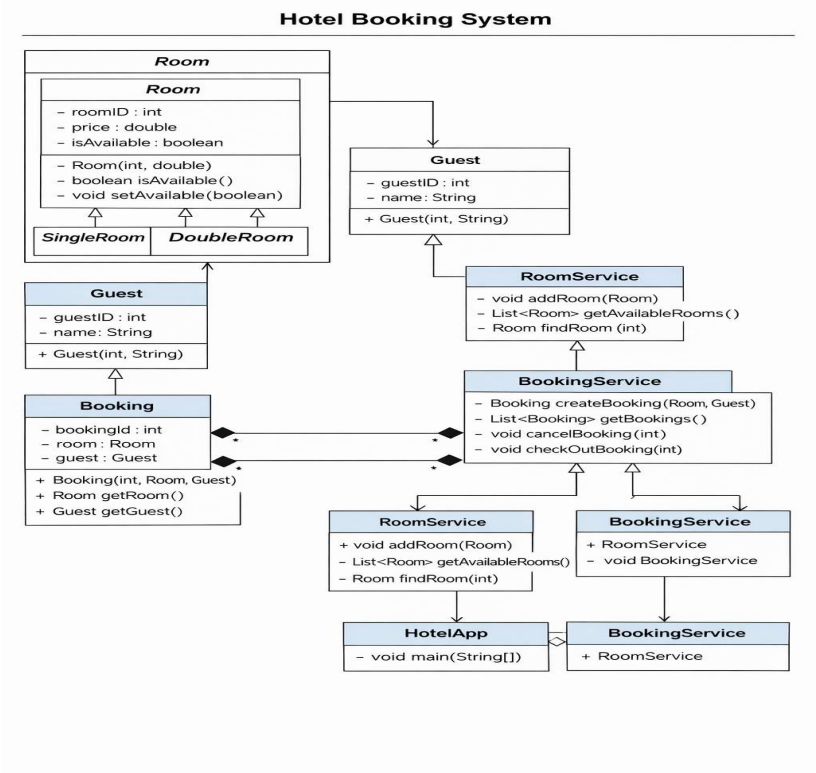
Managing hotel bookings manually is inefficient and prone to errors, especially when dealing with multiple rooms and customers. Common issues include double booking of rooms, incorrect tracking of room availability, and difficulty in maintaining guest records. As the number of customers increases, managing bookings manually becomes time-consuming and unreliable.

The proposed Hotel Booking System provides an automated solution using Java. The system allows users to manage room details, check availability, create bookings, and handle guest information efficiently. It uses Object-Oriented Programming principles such as abstraction, inheritance, polymorphism, and exception handling to ensure modularity and maintainability.

By separating different functionalities into model, service, and main layers, the system improves code organization and scalability. This solution reduces human errors and improves overall efficiency in hotel management.

3. System Design:

3.1 UML Class Diagram:



3.2 Description of Major Classes:

1. **Room:** Represents a general room with attributes like room ID, price, and availability status
2. **SingleRoom, DoubleRoom, SuiteRoom:**
   These classes extend the Room class and define different room categories.
3. **Guest:**
   Stores guest-related information such as name and ID.

4. **Booking:**
   Represents a booking by associating a Guest with a Room.
5. **RoomService:**
   Handles operations related to rooms such as adding rooms and checking availability.
6. **BookingService:**
   Responsible for booking logic including validation, booking creation, and managing bookings.
7. **HotelApp:**
   Main class that acts as the entry point and interacts with the user via console.

3.3 Package Structure:

The project follows a modular package structure to organize code efficiently.

Main Package (Main)

- The model package contains the core data classes such as Room, Guest, and Booking, which represent the main entities of the system.
- The service package includes classes like RoomService and BookingService that handle business logic such as room management and booking operations.
- The exception package contains custom exception classes such as RoomNotAvailableException and InvalidDateException to manage errors effectively.
- The main package includes the HotelApp class, which acts as the entry point of the application and handles user interaction through a console-based interface.
- This modular package structure ensures better code organization, improves readability, and makes the system easier to maintain and extend.

3.4 Interaction Flow:

- User interacts with the system through **HotelApp (console interface)**
- **HotelApp sends request to RoomServie/BookingService**
- **Service layer performs operations like checking availability and creating bookings**
- **Model classes (Room,Guest,Bookin) store and manage data**
- **Result is Displayed back to the user**

4. OOPs Concepts Demonstrated:

a. Abstraction:

Where: Room class

Why: To define a general structure for all room types without exposing implementation details.

Snippet:

```
abstract class Room {
int roomId;
double price;
boolean isAvailable;
}
```

Explanation: Room acts as a blueprint for all room types, ensuring consistency across subclasses.

b. Inheritance:

Where: Room --> SingleRoom, DoubleRoom, SuiteRoom

Why: To reuse common properties and reduce code duplication.

Snippet:

```
class DoubleRoom extends Room {
  }
```

Explanation: All room types inherit basic attributes from Room and can have their own specific behavior.

c. Polymorphism:

Where: **:** Handling different room types

Why: To allow flexibility in handling different room objects

Snippet:

```
Room room = new SuiteRoom();
```

Explanation: A Room reference can refer to any specific room type, enabling dynamic behavior..

d. Exception Handling:

Where: BookingService

Why: To prevent invalid operations like booking an already booked room.

Snippet:

```
if(!room.isAvailable()) {
        throw new RoomNotAvailableException();
        }
```

Explanation: Custom exceptions ensure controlled error handling and improve system reliability.

5. Implementation Highlights:

1. **Room Availability Check**
   ```
   if(room.isAvailable()) {
      // proceed with booking
   }
   ```
   Justification: Ensures that only available rooms can be booked, preventing conflicts.

2. **Booking Creation**
   ```
   Booking booking = new Booking(room, guest);
   ```
   Justification: Connects guest and room into a single booking entity.

3. **Exception Handling**
   ```
   throw new RoomNotAvailableException();
   ```
   Justification: Prevents duplicate bookings and ensures system correctness.

4. **Room Storage**
   ```
   rooms.add(room);
   ```
   Justification: Stores rooms dynamically for further operations.

6. Testing & Error Handling:

- **Test Cases:**

I. Add room successfully.
II. Book available room.
III. View booking details.

- **Edge Cases:**

  I. Booking already booked room
  II. No rooms available.
  III. Invalid room selection

- **Failure Scenarios:**

I. Attempt to book unavailable room → Exception.
II. Invalid user input

7. Git Workflow:

1. Project Initialization

- Created project structure and main class (HotelApp)
- Set up packages: model, service, exception

2. Model Layer Implementation

-implemented core classes: Room, Guest, Booking
-Added subclasses: SingleRoom, DoubleRoom, SuiteRoom

3. **Service Layer Development**

-Created RoomService for managing rooms
-Created BookingService for handling booking operations

4**. Business Logic Implementation**

-Added room availability checking
-Implemented booking creation logic
-Linked Guest and Room through Booking

5. **Exception Handling**

-RoomNotAvailableException
-InvalidDateException
-Integrated exception handling in booking process

6. **Testing & Debugging**

-Tested booking scenarios
-Fixed issues like duplicate booking and invalid inputs

7. **Final Refinement**

-Improved code structure
-Cleaned unused code
-Ensured proper output formatting

The project was developed in a step-by-step manner using version control practices. Each stage of development focused on a specific component, starting from the basic structure to implementing core functionalities like booking and exception handling.

The modular approach allowed independent development of model and service layers, making the system easier to debug and maintain. Regular testing and incremental improvements ensured that errors were identified early and resolved efficiently.

8. Conclusion & Future Scope:

The Hotel Booking System successfully demonstrates the practical application of Object-Oriented Programming concepts in developing a real-world software solution. The system automates essential hotel operations such as room management, booking, and cancellation, thereby reducing manual effort and minimizing errors. By incorporating features like customer management, data persistence through file handling, and multithreading for background processing, the system ensures efficiency, reliability, and improved performance.

The use of a modular structure and separation of concerns enhances code readability, maintainability, and scalability. Additionally, the integration of a user-friendly interface using Java Swing improves the overall user experience. This project not only strengthens the understanding of core OOP principles but also provides a solid foundation for building more advanced and scalable applications in the future.

Future Scope:

- The system can be enhanced by integrating a graphical user interface (GUI) to improve user interaction.
- The application can be extended by adding database integration to ensure permanent data storage.
- It can be developed into a web-based system to enable online room booking.
- A payment gateway can be integrated to allow secure online transactions.
- An admin dashboard can be added for efficient management of hotel operations.
- Advanced features like room filtering and search functionality can be implemented to improve usability.
- A notification system can be introduced to provide booking confirmations and alerts.

1. **Copy of the initial project proposal:**

Project Proposal Document

1. Project Title

Hotel Booking System

2. Problem Statement

Managing hotel reservations manually is error-prone, time-consuming, and difficult to scale. Guests face confusion over room availability, pricing, and booking confirmation, while hotel staff struggle to track occupancy, cancellations, and payments in real time. This project addresses those challenges by building a well-structured, object-oriented Hotel Booking System that models real-world entities such as guests, rooms, bookings, and payments — demonstrating clean design, proper abstraction, and robust error handling.

3. Target User

Hotel front-desk staff and administrators who need to manage room inventory, process guest check-ins/check-outs, and handle booking records through a console-based interface.

4. Core Features

- Search and view available rooms by type (Single, Double, Suite)
- Book a room for a guest with date validation and conflict detection
- Cancel an existing booking with status update
- Check-in and Check-out management with bill generation
- View all bookings for a specific guest
- Admin panel: view occupancy summary and room status
- Persistent storage of bookings using file I/O (optional: CSV or serialized objects)

5. OOP Concepts to be Used

Abstraction:-
Abstract class **Room** defines common room properties (roomNumber, pricePerNight, isAvailable) and an abstract method **getRoomDetails()**. Concrete subclasses (SingleRoom, DoubleRoom, SuiteRoom) implement this method with type-specific details.

Inheritance:-
SingleRoom, **DoubleRoom**, and **SuiteRoom** all extend the abstract **Room** class, inheriting shared attributes while overriding behaviour where needed.

Polymorphism:-
A **List<Room>** holds all room objects. When iterating to display details or calculate pricing, the overridden **getRoomDetails()** method is called polymorphically — each subclass responds differently without the calling code needing to know the exact type.

2. **Copy of the review mail received (with APPROVED_MINOR)**

PCCCS495 Proposal Review – APPROVED_MINOR

S

soumadip.biswas@ieminternational.ai
soumadip.biswas@ieminternational.ai

To You Priyam.Dey2024@iem.edu.in
Tuesday, 10 March at 16:02

Reply-To: soumadip.biswas@ieminternational.ai

Dear Priyam Dey ,

Project: Hotel Booking System
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal clearly articulates a real-world problem and provides a practical, well-scoped solution. The chosen features are relevant and directly address the stated challenges. The explicit mention of key OOP concepts (Abstraction, Inheritance, Polymorphism, Exception Handling, Collections) demonstrates a strong understanding of the project's core requirements. The layered architecture (Model, Service, UI) is a solid choice for promoting separation of concerns, testability, and extensibility, which are critical for good object-oriented

: